Linux Assignment

Name: Muhammad Ibtisam Afzal

A. Directory Navigation

Commands used:

```
pwd
cd ~
mkdir Linux_Lab
cd Linux_Lab
pwd
ls -la
```

Explanation: First, I used `pwd` to check my current location. Then I moved to my home folder using `cd ~`. I created a folder called `Linux_Lab` and entered it. Finally, I checked the folder path and listed its contents using `pwd` and `ls -la`. The `-la` option also shows hidden files and details.

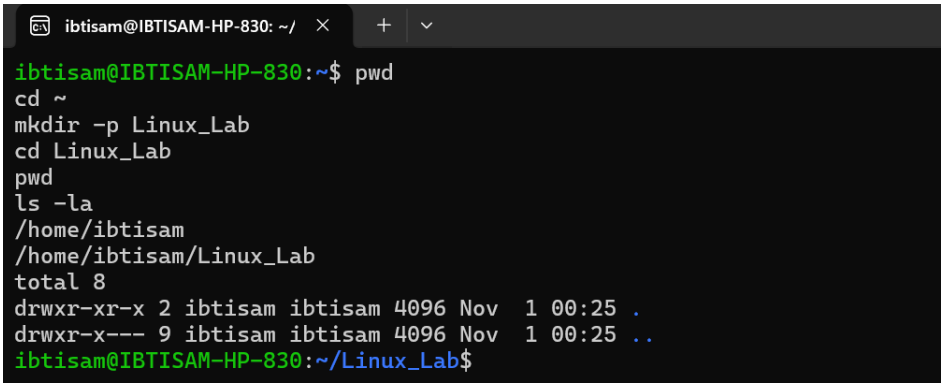


Figure 1: Creating and opening the Linux_Lab folder.

B. File Creation and Reading

Commands used:

```
touch notes.txt commands.txt script.sh
ls -l
cat notes.txt
head -n 5 notes.txt
more notes.txt
less notes.txt
```

Explanation: The `touch` command creates empty files. If a file already exists, it updates its timestamps. I used it to create three files. The `cat` command displays the complete file, while `head -n 5` shows only its first five lines. The `more` command displays text one screen at a time. The `less` command does the same but also allows moving backward and searching. Press `q` to exit `more` or `less`. The files are empty at first, so I can read them again after adding text.

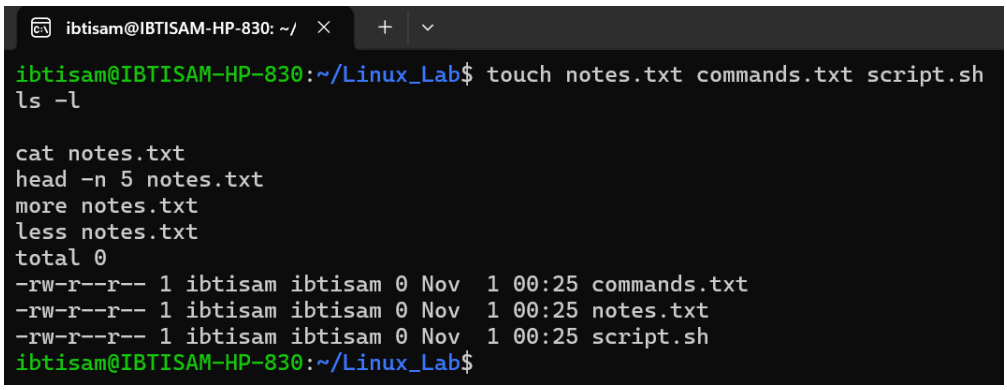


Figure 2: Files created in the coursework folder.

C. File Editing and Aliases

Commands used:

```
nano notes.txt
alias ll='ls -l'
ll
file notes.txt
file script.sh
```

Explanation: I opened `notes.txt` in the `nano` editor and typed some notes. To save, I pressed `Ctrl+O` and `Enter`, then `Ctrl+X` to exit. I created an alias called `ll`, which is a shorter way of writing `ls -l`. Aliases save time when we use the same command often. This alias works for the current terminal session. The `file` command tells us what type of content a file contains.

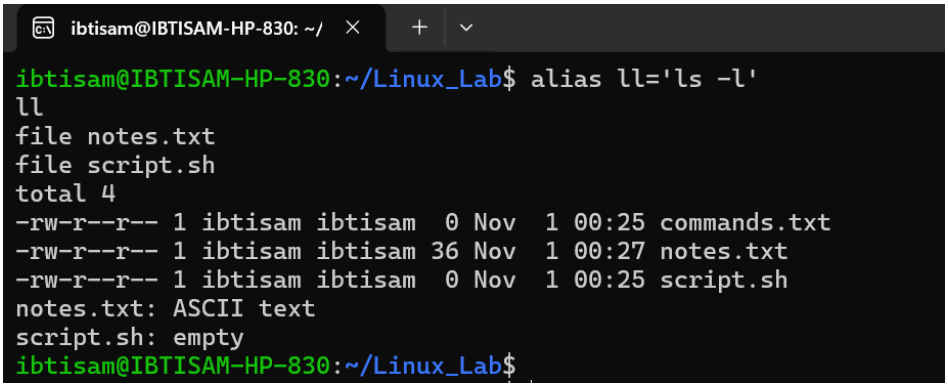


Figure 3: Editing notes and checking the alias and file types.

D. File Permissions

Commands used:

```
ls -l script.sh
chmod 644 script.sh
ls -l script.sh
chmod 755 script.sh
ls -l script.sh
```

Explanation: The permissions `-rw-r-r-` mean that the owner can read and edit the file, while the group and other users can only read it. The first dash shows that it is a regular file. I used `chmod` to change the permissions. With `644`, the permissions are `rw-r-r-`. With `755`, they become `rwxr-xr-x`, meaning the owner can read, write and run the file, while others can read and run it. In these numbers, read is 4, write is 2 and execute is 1.

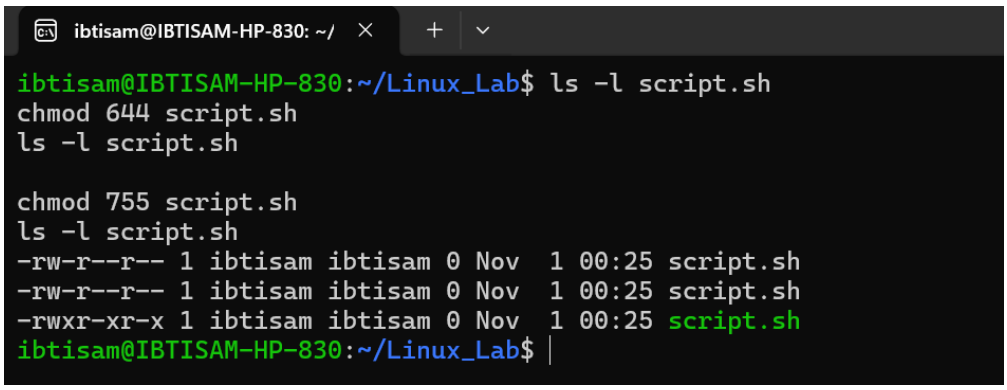


Figure 4: Checking file permissions before and after chmod.

E. Shell and Process Management

Commands used:

```
echo "\$SHELL"
ps aux
ps -p 1234
```

Command asked in the question (not executed):

```
kill 1234
```

Explanation: A shell lets us type commands and interact with Linux. Bash is a commonly used shell with extra features for writing scripts. SH is a simpler shell interface often used for basic scripts. On Debian, `sh` usually runs `dash`. The `ps aux` command shows running processes. Every process has a PID (Process ID), which is its identification number. The `kill 1234` command asks the process with PID 1234 to stop. We should check the process first because stopping the wrong one may close an important program or service. The `echo "$HOME"` command shows the configured login shell.

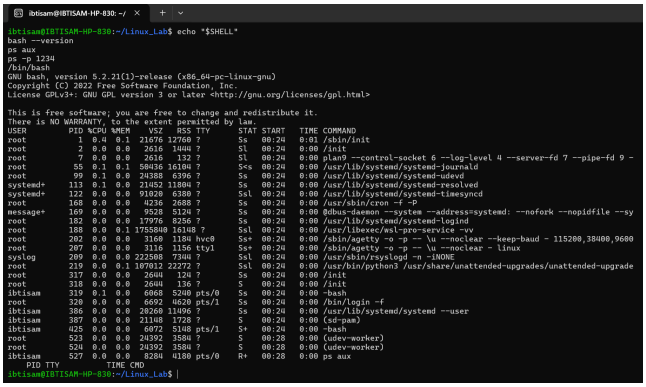


Figure 5: Viewing shell information and running processes.

F. Package Management and System Updates

Commands used:

```
dpkg -l
sudo apt update
apt list --upgradable
sudo apt upgrade --simulate
```

Upgrade command (only run if permitted):

```
sudo apt upgrade
```

Explanation: The `dpkg` tool manages Debian software packages, and `dpkg -l` shows the package list and their status. The `apt update` command refreshes information about available software updates. The `apt upgrade` command installs available upgrades. The `sudo` command gives the required administrator permissions for system changes. I can use `-simulate` to check the proposed upgrade without installing anything. Updating without checking first can sometimes cause software problems or interrupt services.

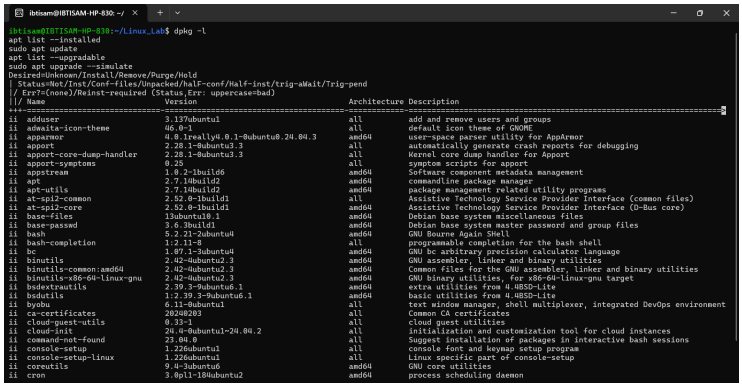


Figure 6: Checking installed packages and available updates.

G. Environment and System Operations

Commands used:

```
echo "\$HOME"
printenv HOME
```

Shutdown and restart commands (not executed):

```
sudo shutdown -h now
sudo reboot
```

Explanation: An environment variable stores information that programs can use. For example, `HOME` stores the path to my home folder, and `echo "$HOME"` displays it. The `sudo shutdown -h now` command shuts down the computer, while `sudo reboot` restarts it. These commands must be used carefully because they can close running programs and cause unsaved work to be lost. I have only written these commands as answers and have not run them.

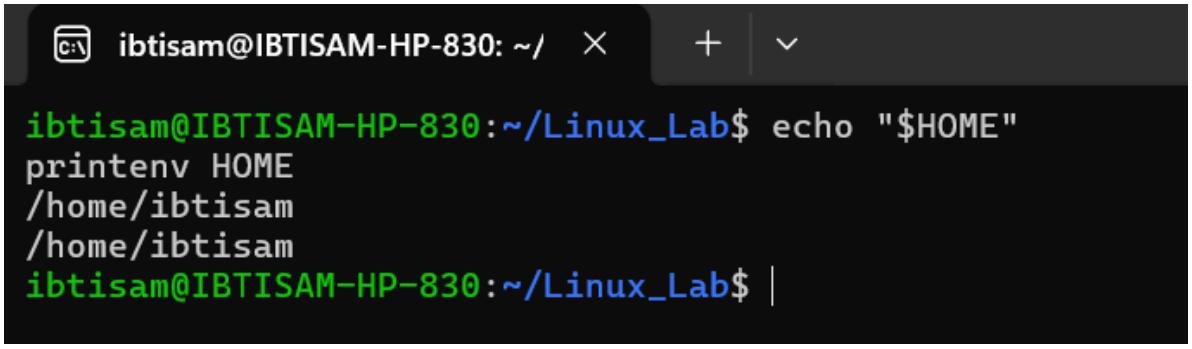


Figure 7: Displaying the HOME environment variable.